

vLLM 面试总结 2

请求全流程、PagedAttention、调度、CUDA Graph、投机解码与专家并行

面向 AI Infra / 推理优化 / vLLM 框架研发岗位

01 请求全流程 API → Scheduler → GPU Prefill / Decode	02 PagedAttention Block Table / Slot Mapping Prefix Cache / Preemption	03 请求调度 Decode Priority Chunked Prefill
04 CUDA Graph Warmup / Capture Padding / Fallback	05 投机解码 EAGLE / EAGLE-3 Verify / Bonus Token	06 专家并行 Router / A2A EPLB / Redundant Experts

版本口径

以 vLLM V1 当前主线、官方文档与原始论文为基础。vLLM 变化很快：类名、默认值、CUDA Graph key、KV block 生命周期和 EP 后端都可能随版本演进；面试回答应先讲稳定的设计逻辑，再补“当前主线实现”。

整理日期：2026-08-10

使用方法与原始问题

答题总公式

先说结论，再沿着五条线展开：①请求状态在哪里；②本轮 Scheduler 给了多少 token budget；③ KV block 如何分配与复用；④ GPU 上执行哪种 forward / graph；⑤通信、计算和端到端 SLO 分别受什么影响。

本册覆盖三张截图中的全部问题。每一题都给出面试结论、实现细节、容易说错的点以及 60 秒口述版。建议先通读最后一章的速查，再回到正文补原理。

问题清单

- 一个 vLLM 请求从输入到最终输出经历什么？
- Prefill 和 Decode 阶段如何进入调度？是否真的有两个独立队列？
- 为什么必须做 Prefill？资源不足时 vLLM 会怎么处理？
- PagedAttention 如何管理显存？
- PagedAttention 与 Prefix Cache 是什么关系？多个公共 block 如何匹配？
- 请求使用的 block 被淘汰或抢占后怎么办？彻底 free 后旧数据如何处理？
- Prefill 和 Decode 谁优先？如何兼顾 TTFT 与 ITL？
- Graph mode 是什么？为什么 warmup？新 shape 如何 padding 或 fallback？
- 投机解码的完整流程、加速原因、接受条件和命中指标是什么？
- EAGLE、EAGLE-2 与 EAGLE-3 的区别是什么？
- 主模型与草稿模型如何生成/验证多个 token？
- Bonus token 是什么？为什么可以直接提交？
- EP 如何选专家、执行专家计算并做负载均衡？
- 多卡场景下 Router、专家权重和逻辑/物理专家映射如何工作？
- 跨卡专家如何通过 All-to-All 发送数据？
- EPLB 如何分配压力？“平衡冷门专家”到底是什么意思？

六个高频纠错

V1 不是简单的“Prefill 队列 + Decode 队列”；Prefix Cache 命中必须是从开头开始的连续前缀；活跃请求引用的 block 不能被缓存淘汰；Target 验证 K 个草稿 token 不是 K 次自回归；Bonus token 来自 Target 而非 Draft；EPLB 不会把 token 随便改路由到冷门逻辑专家。

01

请求全流程与 Prefill / Decode

从 HTTP 请求、统一调度到逐 token 流式返回

vLLM V1：一个请求从入口到流式输出的完整链路



V1 的关键不是“先固定组一个 batch”，而是每个 engine step 重新选择请求与 token 数；prefill、decode、prefix cache、投机解码都被统一成 token budget 问题。

图 1: vLLM V1 的请求生命周期与每步 continuous batching。

Q | vLLM 学习过吗？请说一下一个请求从输入到最后输出结果经历的过程。

面试结论

入口层把文本转换成内部 Request；EngineCore 的 Scheduler 在每个 step 决定“哪些请求、本轮算几个 token”，KV Manager 完成前缀查询和 block 分配；Executor/Worker/ModelRunner 执行 Prefill 或 Decode，Sampler 产生 token，输出层 detokenize 并流式返回，直到停止条件触发后释放资源。

1.1 完整链路：按实际执行顺序回答

1. API Server 接收请求：校验模型名、sampling parameters、max_tokens、stop、logprobs、LoRA、多模态输入等；Chat API 还会先应用 chat template。

2. Tokenizer / Input Processor: 文本转 token IDs; 多模态输入经过 processor/encoder 预处理并形成占位 token、特征引用或 encoder cache key。
3. 构造内部 Request: 保存 prompt token、全部已生成 token、采样状态、优先级、到达时间、最大长度、结构化输出状态等。
4. Request 送入 EngineCore: 新请求先进入 waiting; EngineCore 持有 Scheduler、KV Cache Manager、结构化解码管理器及可选 KV Connector。
5. Scheduler 每个 engine step 重新组批: 根据 token budget、max_num_seqs、优先级/FCFS、KV 空间、encoder budget 和 speculative lookahead, 产生 SchedulerOutput。
6. Prefix Cache 查询与 KV 分配: 计算最长可复用前缀, 给未命中 token 和投机 lookahead 分配物理 block IDs; 必要时推迟、分块或抢占请求。
7. Executor 将 SchedulerOutput 广播/分发给每个 Worker; TP/PP/DP/EP 的设备进程保持对应的请求、block table 与通信组状态。
8. ModelRunner 更新 persistent batch: 添加/删除/重排请求, 准备 input_ids、positions、block tables、slot mappings、seq lens、采样元数据和 CUDA Graph descriptor。
9. Prefill: 对尚未计算的 prompt tokens 做前向, 写入 K/V; 最后一个 prompt 位置的 logits 用于采样首个输出 token。Chunked Prefill 时可能跨多个 step 完成。
10. Decode: 下一轮把最新 token 作为 query, 读取历史 paged KV, 写入新的 K/V, 得到下一个 token; 所有运行中请求在每步动态混批。
11. Output Processor: 处理 logits processor、采样、logprobs、stop string/EOS/长度限制; 把 token IDs 交给 detokenizer 并以增量事件流回 API Server。
12. 结束与清理: 请求完成、取消或失败时, 释放/缓存 KV blocks、清理 runner state 和可选远端 KV 状态; API Server 返回最终 usage 与 finish_reason。

1.2 各层职责一句话

层	核心职责	面试里最重要的变量
API / AsyncLLM	协议、模板、tokenize、异步流式输出	请求到达时间、参数、取消信号
EngineCore	持有请求状态并驱动 step	waiting/running、完成事件
Scheduler	决定本轮每请求计算 token 数	token budget、max_num_seqs、优先级
KV Manager	查 prefix、分配/释放/引用 block	block IDs、refcount、hash、free queue
Executor	跨进程/跨节点编排设备执行	rank、collective、worker RPC
Worker	单设备权重、KV、通信和运行时状态	device、model shard、cache tensor
ModelRunner	把调度结果变成 GPU 输入并 forward	block table、slot mapping、graph key

层	核心职责	面试里最重要的变量
Sampler / Output	从 logits 到 token 与文本	temperature、top-k/p、stop

面试官真正想听什么

不要只背类名。必须说清：SchedulerOutput 是 CPU 控制面的“本轮执行计划”；ModelRunner 把它物化成静态 GPU buffers；Attention kernel 依靠 Block Table / Slot Mapping 访问 paged KV；Sampler 产出 token 后再驱动下一步。

60 秒口述版

一个请求先在 API 层做模板、tokenize 和参数校验，形成内部 Request 进入 EngineCore。Scheduler 每个 step 根据 token budget、KV blocks 和优先级动态选请求，并由 KV Manager 做前缀命中和块分配。Executor 把计划发到各 Worker，ModelRunner 更新 persistent batch、block table、slot mapping 并执行 prefill 或 decode。Sampler 产生 token，detokenizer 流式返回；未结束的请求继续参加下一轮 continuous batching，结束后释放 KV 和运行时状态。

Q | Prefill 和 Decode 阶段与两个请求队列怎么调度？

面试结论

概念上可以把“尚未完成 prompt 的请求”和“正在生成的请求”分开理解，但 vLLM V1 使用统一 Scheduler：它维护 waiting/running 等请求状态，并用 {request_id: num_tokens} 给每个请求分配本轮 token 数，不要求 Prefill 与 Decode 完全分成两个互斥批次。

1.3 V1 为什么叫统一调度

- Prefill 与 Decode 的共同抽象都是“从 num_computed_tokens 往后再计算 N 个 token”。
- 一个 batch 可以同时含 decode token、chunked prefill token、encoder token 和 speculative token。
- Scheduler 的输出是每请求的 num_scheduled_tokens，而不是简单的 prefill_batch / decode_batch 二选一。
- 请求层面的先后仍受 FCFS 或 priority policy 影响；阶段层面的资源分配还受 decode 优先、长 prefill 限额等机制影响。

1.4 Prefill 与 Decode 的计算特征

维度	Prefill	Decode
输入形态	一次处理 prompt 的多个 token，可被 chunk	通常每请求 1 token；投机解码时为 1+K

维度	Prefill	Decode
主要目标	构建 KV，并得到首 token logits	持续产生后续 token
硬件特征	更像大矩阵乘，batch/sequence 足够时偏 compute-bound	反复读大权重和历史 KV，常偏 memory-bandwidth-bound
用户指标	决定 TTFT	决定 ITL / TPOT
调度策略	用剩余 token budget，过长可切块	通常优先以保护交互延迟

Q | 为什么要有 Prefill？请求调度时如果资源不足，vLLM 会做哪些处理？

面试结论

Prefill 是因果 Transformer 的数据依赖：在生成第一个新 token 前，必须为整段 prompt 计算每层 K/V 和末位置 logits。资源不足时，vLLM 会按程度依次采用 prefix reuse、chunked prefill、延后 admission、预留 watermark、抢占并重算，或在请求永远无法容纳时直接报错/拒绝。

1.5 为什么不能跳过 Prefill

Decode 第 t 个位置的注意力要读取此前所有有效 token 的 K/V。Prompt 的 K/V 只有经过模型前向才能得到，所以 Prefill 不是“系统为了方便划分的阶段”，而是计算依赖本身。Prefix Cache 或远端 KV 命中只是把已经算过的 Prefill 结果拿来复用，并没有消除这段 K/V 的存在。

1.6 资源不足时的动作阶梯

情况	典型动作	代价 / 目的
本轮 token budget 不够	缩小某请求本轮 token 数；长 prompt 做 chunked prefill	不释放已有 KV，延后完成 Prefill
max_num_seqs 已满	新请求留在 waiting	保护当前 batch，增加排队 TTFT
KV blocks 暂时不足	延后新请求；保留 watermark headroom	减少反复抢占和缓存抖动
运行中请求挤占 KV	按策略 preempt 某请求，释放其 blocks，回到 waiting	恢复时需 prefix hit 或 recompute
启用 KV offload/connector	从 CPU/远端缓存加载或保存 KV	换显存，增加传输和同步
单请求本身永远放不下	长度校验失败或不可调度，终止请求	避免无限等待/死循环

V1 抢占的关键点

基础 V1 路径通常释放该请求的 GPU KV blocks，把请求标记为 PREEMPTED 并重新排队；恢复时基于仍存在的连续 prefix cache 命中重建状态，缺失部分重新 Prefill。旧 V0 文档里的 GPU↔CPU swap 不是理解当前 V1 的默认主线。

60 秒口述版

Prefill 先算出 prompt 的各层 KV。V1 统一按 token budget 调度，通常 Decode 优先，剩余预算做 chunked prefill。KV 紧张时先等待或限流，必要时抢占并释放 block；恢复时复用最长 prefix、重算缺失部分，单请求始终放不下则拒绝。

02

PagedAttention、Prefix Cache 与抢占恢复

物理显存池、连续前缀命中和 block 生命周期

PagedAttention：共享、抢占、恢复与物理块复用

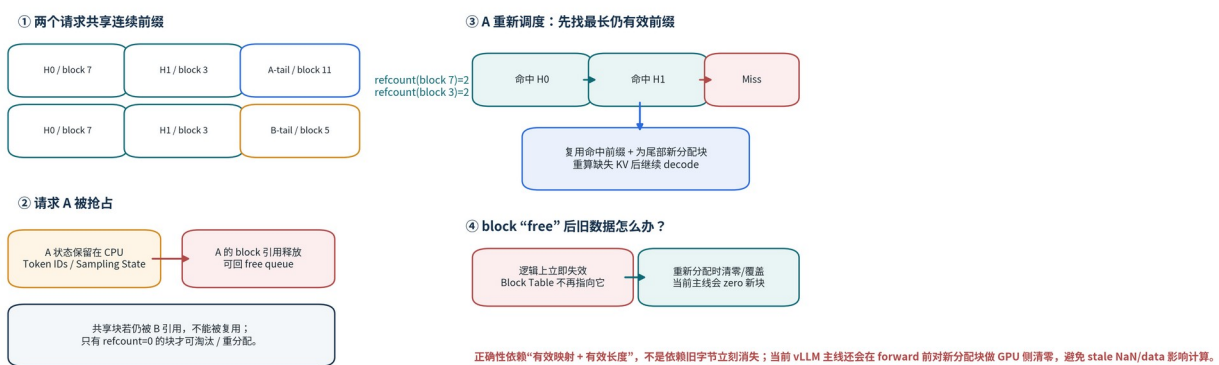


图 2：共享前缀、请求抢占、恢复重算与新块清零。

Q | PagedAttention 如何管理显存？

面试结论

启动时把可用于 KV 的显存预分配为固定大小的物理 blocks；每个请求只维护逻辑 block 序列，Block Table 将逻辑块映射到任意物理 block，Slot Mapping 指定本轮新 token 写入哪个槽位。KV Manager 在 CPU 侧维护 free queue、refcount、prefix hash 与分配状态，Attention kernel 按这些映射直接读写。

2.1 为什么要分页

传统做法若给每个请求预留最大连续 KV，会产生巨大的预留浪费和外部碎片。PagedAttention 借鉴虚拟内存：逻辑序列连续，物理显存不要求连续；请求增长时按 block 增量分配，结束时按 block 归还。这样 continuous batching、不同长度请求混跑、prefix sharing 和抢占才可高效实现。

2.2 三个关键数据结构

结构	含义	谁使用
Physical KV Pool	按层/组预分配的 K/V tensors, block 是分配单位	Worker / Attention backend
Block Table	request logical block index → physical block id	ModelRunner 生成, Attention 读取历史 KV
Slot Mapping	本轮每个新 token → 具体物理写入槽位	Cache write / Attention kernel
Seq Lens / Query Start	每个请求有效长度、flatten 后起始位置	Attention mask 与边界控制
Block metadata	refcount、hash、free-list 指针、group id	CPU KV Manager

2.3 地址关系

```

logical_block = position // block_size
block_offset  = position % block_size
physical_id   = block_table[request][logical_block]
slot          = physical_id * block_size + block_offset

```

真实实现还要考虑层、KV head、head dimension、KV cache group、DCP/CP、不同 attention backend 的物理布局，但面试里抓住“逻辑块到物理块的间接映射”即可。

2.4 KV 容量如何估算

```

bytes_per_token ≈ 2 × num_layers × num_kv_heads × head_dim × bytes_per_element
KV_pool_tokens ≈ available_KV_bytes / bytes_per_token

```

其中 2 对应 K 和 V。GQA/MLA、FP8 KV、滑动窗口、混合 Mamba state 和 DCP 会改变公式，但这个估算能快速判断瓶颈是权重还是 KV。

60 秒口述版

PagedAttention 把 KV 显存预分配为固定物理 blocks。请求看到的是连续逻辑块，Block Table 把它们映射到离散物理块，Slot Mapping 给新 token 指定写入位置。CPU 侧 KV Manager 管 free list、refcount、hash 和生命周期，GPU Attention kernel 只按映射读历史 KV、写新 KV。它用少量间接寻址开销换取低碎片和动态批处理。

Q | PagedAttention 和 Prefix Cache 的关系是什么？两个请求有多个公共 block 时如何匹配？

面试结论

PagedAttention 提供“物理 block 可被多个逻辑请求引用”的机制；Prefix Cache 决定哪些已计算 block 可以按内容复用。vLLM 对前缀按链式 hash 查找，命中必须从第 0 块开始连续延伸，直到第一个 miss；不是在两个请求的 block 集合里任意找相同块。

2.5 Prefix Cache 的链式哈希

```
H_0 = hash(seed, block_tokens_0, extra_keys_0)
H_i = hash(H_{i-1}, block_tokens_i, extra_keys_i)
extra_keys can include: LoRA identity, multimodal hashes, cache_salt, prompt-embed hash
```

- 父 hash 把“前面所有 token”编码进当前块，避免相同局部 token 出现在不同上下文时误命中。
- 传统主线只把完整块加入 prefix cache；当前主线正在支持 hash 粒度与物理 block 粒度解耦，用于 hybrid/Mamba 的细粒度命中，但物理分配单位仍是 block。
- 命中时，新请求的 Block Table 直接引用已有 physical IDs，并增加 refcount；只计算第一个 miss 后的尾部。

2.6 多个公共 block 的匹配例子

```
Request A blocks: H0, H1, H2, H3
Request B hashes: H0, H1, Hx, ...
lookup result: reuse H0 + H1, stop at Hx
NOT: continue searching for a later block equal to H3
```

原因是 Transformer 的 KV 取决于完整前缀。只要中间一个 token 不同，后续位置的 hidden states 与 K/V 通常全部改变，所以只能复用最长连续公共前缀。

2.7 共享与 Copy-on-Write

- 完整共享块只读：多个请求 refcount>1，任何请求都不能改写已有有效 token。
- 请求继续增长时，通常给尾部新分配 block；若命中边界位于物理块内部，当前细粒度方案需要 copy-on-write 后再追加。
- 缓存淘汰只选择 refcount=0 的缓存块；正在被活跃请求引用的块不能被 Prefix Cache LRU 淘汰。

60 秒口述版

PagedAttention 让多个逻辑请求能引用同一个物理 KV block，Prefix Cache 则用内容 hash 判断能否复用。每块 hash 包含父块 hash、当前 token 和 LoRA/多模态等额外键。查找从开头逐块进行，命中 H0、H1 后遇到 H2 miss 就停止，只复用最长连续前缀；命中的 block refcount 加一，尾部重新分配和计算。

Q | 一个请求分配过被淘汰的 block 后需要做什么？重新启用会怎样？彻底 free 后旧数据怎么处理？

面试结论

先区分三件事：缓存淘汰只作用于零引用块；运行请求在显存压力下可能被 preempt 并释放自己的 KV；普通 free 只是结束所有权。被抢占请求恢复时重新查询最长有效 prefix，缺失 KV 重新计算。物理 block 逻辑失效后不再可读；当前 vLLM 主线在新分配块进入 forward 前还会做 GPU 侧清零。

2.8 三种“块没了”不要混为一谈

事件	能否发生在活跃引用块	请求状态	恢复方式
Prefix Cache eviction	不能；要求 refcount=0	请求不受影响	以后新请求不再命中该缓存块
Request preemption	可以主动释放该请求的引用	PREEMPTED，回 waiting	命中仍存 prefix + recompute 缺失部分
Request finish/free	该请求的 refcount--	完成/取消	无需恢复；块可缓存或回 free queue
External KV invalidation	取决于 connector / lease	调整 num_computed_tokens	回退到最长有效前缀再重算

2.9 被抢占请求重新启用的状态机

1. CPU 侧仍保留 prompt/output token IDs、sampling state、stop 状态等逻辑请求信息。
2. Scheduler 将其重新放入 waiting；重新 admission 时查询本地/远端可用 KV。
3. 把 num_computed_tokens 回退到最长仍有效的连续前缀。
4. 为缺失区间重新分配 blocks，重新 Prefill 已知 token 序列以重建 KV。
5. 恢复到原生成位置后继续 Decode；输出 token 不会因为抢占而重新随机生成。

2.10 Free 后旧字节的安全语义

- 逻辑安全：Block Table 和 refcount 先失效，Attention 只读取有效映射和 seq_len 范围，旧内容立刻不可达。
- 物理处理：历史实现常在下次写入时覆盖；当前主线会收集 newly allocated block IDs，并在 forward 前通过 GPU/Triton zeroer 清零，以防 stale NaN/data 污染 Attention 或状态模型。
- 多租户安全还要关注 hash collision、cache_salt、跨请求 timing side channel 与远端 KV 权限；“内存复用正确”不等于“租户隔离自动完美”。

60 秒口述版

先纠正一个前提：活跃请求正在引用的 block 不会被 Prefix Cache 淘汰。显存不足时是 Scheduler 抢占请求，释放它的 KV block，并保留 CPU 侧 token 和采样状态。请求恢复后先命中最长仍存在的 prefix，缺失部分重新 prefill。block free 后映射立即失效，旧数据不会再被 Attention 读取；当前主线还会在块重新分配后、forward 前做 GPU 侧清零。

03

请求调度与 CUDA Graph

Decode 优先、Chunked Prefill、Warmup、Padding 与回退

Q | Prefill 和 Decode 的调度关系是什么？哪个有更高优先级？

面试结论

在当前 vLLM 的 chunked prefill 策略下，通常先调度所有可运行的 Decode，再用剩余 max_num_batched_tokens 预算安排 Prefill；过长 Prefill 被切块。这样优先保护 ITL/TPOT，但请求层面仍由 FCFS 或 priority policy、KV 空间和公平性规则共同决定。

3.1 为什么通常 Decode 优先

- 一个 Decode 请求本轮通常只需要 1 个 token，批量加入成本可控，却直接决定所有在线用户的下一 token 延迟。
- 长 Prefill 若不切块，可能独占一次大 forward，让几十个 Decode 请求停顿，造成明显 ITL tail spike。
- Prefill 常更 compute-bound，Decode 常更 memory-bound，混批能提高资源利用率；剩余 token budget 给 Prefill 可减少空泡。
- Decode 优先不是绝对“饿死 Prefill”：系统需要 max_num_batched_tokens、long_prefill_token_threshold、请求优先级和 admission 控制来保证进展。

3.2 三层优先级不要混淆

层次	决定什么	常见规则
请求排序	哪个 request 先被考虑	FCFS；或显式 priority，值更小/更高优先级，arrival time 破同分
阶段预算	本轮 token budget 先给谁	Chunked prefill 下 Decode 通常先拿预算
资源可行性	即使优先也能否执行	KV block、max_num_seqs、encoder budget、graph/backend 约束

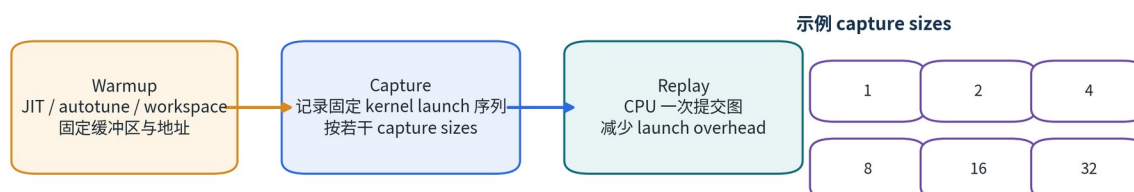
3.3 TTFT 与 ITL 的权衡

调大 / 调小	收益	风险
更小 <code>max_num_batched_tokens</code>	Decode 更容易穿插, ITL 更稳	Prefill 被切得更碎, TTFT 和总吞吐可能变差
更大 <code>max_num_batched_tokens</code>	Prefill 吞吐更高、TTFT 可能更好	大批 Prefill 与 Decode 竞争, ITL tail 可能上升
限制长 Prefill 每步 token	防止单个长 prompt 霸占预算	长请求完成 Prefill 更慢
提高请求 priority	关键请求更早获得预算	低优先级请求可能饥饿, 需要 aging/限额

60 秒口述版

当前 V1 是统一 token scheduler, 但启用 chunked prefill 后, 阶段上通常优先把 pending decodes 放进 batch, 再用剩余 `max_num_batched_tokens` 调度 prefills; 放不下的长 prefill 就切块。原因是 decode 每请求只算少量 token, 却直接影响所有用户的 ITL。请求之间仍由 FCFS/priority 和 KV 资源决定, 所以“Decode 优先”不是唯一排序规则。

CUDA Graph : Warmup、Capture Key、Padding 与 Fallback



运行时 shape (主要是 `num_tokens / num_requests`)



Padding 行使用静态输入 buffer; Block Table 的空行填 null block, Slot Mapping 填无效槽位 (如 -1), 保证 dummy token 不污染 KV。

Graph mode $\neq$ torch.compile: 前者重放 GPU launch 图, 后者优化/融合计算图; vLLM 可以组合二者。

图 3: CUDA Graph 从 warmup/capture 到 runtime padding 与 fallback。

Q | Graph mode 是什么？为什么要先 warmup？warmup 后遇到未命中的新 shape 会如何 padding？

面试结论

这里的 graph mode 通常指 CUDA Graph：把固定的一组 GPU kernel launches 和静态 buffer 地址捕获后反复 replay，降低 CPU/Python 与 kernel launch 开销。Warmup 先完成 JIT、Triton/CuBLAS 选择、workspace/allocator 初始化并稳定地址；运行时 shape 被向上 padding 到可兼容的 capture size，超出最大范围或 key 不兼容则回退到 piecewise/eager。

3.4 CUDA Graph 与 torch.compile 的区别

机制	解决的问题	约束
torch.compile / Inductor	图级优化、算子融合、生成高效 kernel	动态 shape、graph break、编译时间
CUDA Graph	一次 replay 提交固定 kernel launch 序列，降低 CPU launch overhead	静态地址、固定 capture key、额外显存
Piecewise CUDA Graph	只捕获可 graph 的片段，动态/不兼容 op 留在图外	边界开销，但灵活性更强
Full CUDA Graph	捕获整个模型 step	要求更严格，特定 attention/通信/LoRA 组合可能不支持

3.5 为什么 Capture 前必须 Warmup

- Triton / CUDA kernel 可能在首次执行时 JIT 编译。
- cuBLAS/cuBLASLt、FlashAttention、MoE kernel 可能首次选择算法或申请 workspace。
- PyTorch caching allocator 与模型自定义 op 可能进行 lazy allocation。
- NCCL/通信后端、sampler、KV zeroing kernel 等也可能首次初始化。
- 若这些动态行为发生在 capture 内，会导致非法分配、错误地址、capture failure 或把一次性开销固化。

vLLM 的配置里可以指定 `cudagraph_num_of_warmups`：前若干次 dummy run 只用于预热，之后才记录 graph。Warmup 不是为了“训练模型”，而是为了让运行时路径稳定、可捕获。

3.6 什么叫一个 CUDA Graph shape / key

模型 hidden size 等权重维度固定，Serving 里最常变化的是 `num_tokens`、`num_requests`、是否为 uniform decode、每请求 query length、是否启用 LoRA/活跃 LoRA 数、attention backend 能力以及某些模型特性。因此 vLLM 不是只按一个整数盲目匹配，而是根据 BatchDescriptor / capture key 选择 FULL、PIECEWISE 或 NONE。

3.7 新 shape 如何 Padding

```
capture_sizes = [1, 2, 4, 8, 16, 32, ...]
actual num_tokens = 13 -> padded graph size = 16
actual num_tokens = 17 -> padded graph size = 32
actual > max_capture_size or incompatible key -> fallback (piecewise/eager)
```

- Padding 是向上取最小可用 capture size，而不是向最近值四舍五入。
- 真实 token 放在静态输入 buffer 前部，尾部 dummy rows 被标记为 padding。
- 空 Block Table 行填 NULL_BLOCK_ID，Slot Mapping 填无效位置，避免 dummy token 写入真实 KV。
- FULL decode 对 num_reqs / uniform query length 更敏感；PIECEWISE 可以放宽部分 key。
- Capture sizes 越密，padding 浪费越少，但启动 capture 时间和 graph 显存占用越大。

3.8 什么时候会回退

情况	处理
num_tokens 超过 max_cudagraph_capture_size	Eager / 非 graph 路径
Attention backend 不支持 full graph	Piecewise 或 NONE
混合 Prefill-Decode 与 FULL_DECODE_ONLY	不走 full graph
LoRA 活跃数不在已捕获 key 中	选择兼容 graph 或 fallback
动态模型特性/多模态路径发生 graph break	图外执行或 eager
捕获路径需要动态内存/地址变化	修复静态 buffer，或禁用该 graph 模式

最容易答错的一句

“遇到任何新 shape 都 padding 到最近 graph”是错的。只能在已配置 capture 范围内、且其余 capture key 兼容时向上 padding；否则必须换成更宽松的 PIECEWISE 或直接 EAGER。

60 秒口述版

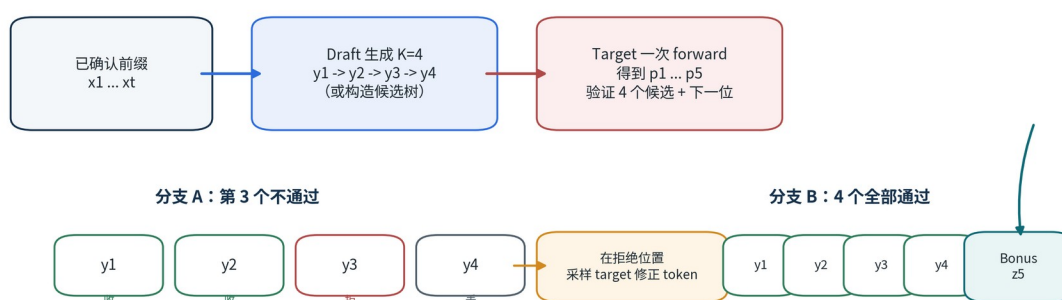
Graph mode 指 CUDA Graph replay：把固定 kernel launch 序列和静态 buffer 地址捕获起来，后续一次提交，主要减少 CPU launch overhead。Capture 前先 dummy warmup，让 JIT、autotune、workspace 和 allocator 都稳定。运行时主要按 num_tokens/num_requests 等 key 匹配，13 个 token 可能 pad 到 16；dummy 行用 null block 和无效 slot。超过最大 capture size 或 LoRA、attention 等 key 不兼容时，不会硬 pad，而是回退 piecewise/eager。

04

投机解码：验证、EAGLE-3 与 Bonus Token

无损加速的接受规则、性能模型和生产边界

投机解码：草稿、并行验证、拒绝修正与 Bonus Token



Greedy：逐位比较 draft token 与 target argmax，接受最长一致前缀。Sampling：第 1 位以 $\min(1, p_i(y_i)/q_i(y_i))$ 接受；首次拒绝时从正部差分布采样修正 token。

Bonus token 不是“额外相信草稿”，而是 target 在同一次验证 forward 中已经计算出的第 $K+1$ 位，因此可直接按 target 分布提交；若 K 个草稿全通过，一轮最多提交 $K+1$ 个 token。

图 4：标准投机采样的两条分支——首次拒绝修正，或全部接受后提交 Bonus Token。

Q | 投机解码的流程是什么？为什么能加速？什么条件下接受草稿 token？命中率大概多少？
EAGLE 和 EAGLE-3 有什么区别？

面试结论

廉价 proposer 先给出 K 个候选，昂贵 Target 用一次多 token forward 并行计算对应分布；按 greedy 一致性或 rejection-sampling 规则接受最长前缀，首次拒绝处用 Target 修正。它把多次昂贵 Decode 合并为一次 Verify，适合 memory-bound、低到中等 batch。接受表现没有统一百分比，应看平均接受长度与端到端 speedup；EAGLE-3 相比 EAGLE 去掉 feature regression 约束、融合低中高层特征，并用 training-time test 缓解多步自回归误差。

4.1 标准投机解码的完整流程

1. 已有 Target 已确认前缀 $x_1 \dots x_t$ ，Draft 模型条件于该前缀。

2. Draft 以较低成本顺序提出 K 个 token $y_1 \dots y_K$ ，并保留每一步草稿分布 q_i ；树方法会提出多条候选路径。
3. Target 把这些候选 token 作为 teacher-forced query，一次 forward 计算 $p_1 \dots p_K$ 以及下一位置 p_{K+1} 。因果 mask 保证每个位置只看前缀。
4. Verifier 从前到后接受 token，遇到第一个失败即停止；后面的候选全部作废。
5. 若首次拒绝发生在 i ，提交此前已接受前缀，并在位置 i 按 Target 修正规则采样一个 token。
6. 若 K 个全部接受，再从 Target 已算出的 p_{K+1} 采样 Bonus token；一轮最多提交 $K+1$ 个 token。
7. 只保留已提交 token 的 KV，拒绝路径上的临时候选 KV 被丢弃/回滚，进入下一轮。

4.2 接受规则

Greedy / temperature=0

```
accept  $y_i$  iff  $y_i == \operatorname{argmax}_x p_i(x)$ 
accept the longest consecutive prefix; at first mismatch use target  $\operatorname{argmax}$ 
```

Sampling / lossless speculative sampling

```
 $\alpha_i = \min(1, p_i(y_i) / q_i(y_i))$ 
accept  $y_i$  with probability  $\alpha_i$ 
on first rejection sample from  $r_i(x) \propto \max(0, p_i(x) - q_i(x))$ 
```

这个接受/修正构造保证最终 token 分布与直接从 Target 自回归采样一致，因此叫 lossless。工程实现可能在数值稳定、top-k/top-p 截断、树验证和并行 sampler 上有所变化，但核心原则不变。

4.3 为什么能加速

- 普通 Decode 每生成 1 token 都要让 Target 读一遍大权重；在小 batch 下常受 HBM 带宽和 launch overhead 限制。
- Verify 一次处理 $K+1$ 个 query positions，能够复用权重读取并提高 GEMM 算术强度。
- Draft 只要远小于 Target，且平均每轮提交多个 token，总 Target forward 次数就减少。

```
rough speedup  $\approx E[\text{committed tokens per round}] \times C_{\text{target\_decode}}$ 
/  $(C_{\text{draft}} + C_{\text{target\_verify}} + C_{\text{metadata}})$ 
```

这只是直觉模型。真实速度还受 batch、context length、KV 带宽、候选树宽度、draft placement、CUDA Graph、MoE A2A 和 sampler 影响。

4.4 什么时候容易负收益

条件	为什么变差
高并发 / 大 batch	Target 本来已接近 compute-bound，多 token verify 不再“白送”
Draft 太大或跨节点	Draft latency、通信和额外 KV 抵消收益
接受长度低	大部分候选被丢弃，verify 计算浪费
长上下文 Attention 很重	K 个候选增加 query×KV 工作量
MoE Target	每轮多 token 可能放大 EP dispatch/combine 通信
采样温度高或任务分布漂移	Draft 与 Target 对齐度下降

4.5 “命中率多少”应该怎么答

没有通用数字

必须先问指标定义：draft token acceptance rate、每位置 acceptance、平均接受长度 τ 、每轮提交 token 数，还是端到端 speedup。不同 K、树宽、温度、模型、任务和 batch 下不能直接横比。

公开结果	指标	如何理解
EAGLE-2 论文	每个 draft-verify cycle 约生成 4–5.5 个 token	跨任务平均接受长度量级，不是统一百分比
EAGLE-3 论文	MT-bench τ 4.05→6.13；GSM8K 4.24→6.23 (8B ablation)	EAGLE-3 相对 EAGLE-2 的任务相关提升
EAGLE-3 HumanEval	最高平均接受长度约 7.5，论文 speedup 最高约 6.5×	代码模板多，容易预测；不是线上保证值
生产部署	看 accepted_length、acceptance_rate、draft/verify latency、P99 TPOT	必须用自己的 prompt、采样与并发压测

面试中可以补一句：若固定 $K=8$ ，平均接受 6 个 draft token，粗略 token acceptance 看起来约 75%，但树候选、Bonus token、提前终止与指标口径会让这个换算不严谨，所以工程上直接报告平均接受长度更可靠。

4.6 EAGLE、EAGLE-2 与 EAGLE-3

方法	Draft 信号与结构	核心改进	面试关键词
EAGLE	用 Target hidden feature + token 预测下一步 feature/token	Feature-level autoregression，降低纯 token draft 的不确定性	预测 feature，轻量 draft
EAGLE-2	沿用 EAGLE draft，构建动态 draft tree	根据候选置信度动态扩展和重排，更好利用固定节点预算	Dynamic Draft Tree

方法	Draft 信号与结构	核心改进	面试关键词
EAGLE-3	直接预测 token distribution; 融合 Target 低/中/高层 feature	移除 feature regression constraint; Training-Time Test 让 draft 见到自身多步预测, 减轻 exposure gap	Feature fusion + TTT

EAGLE-3 的本质

不是“把 EAGLE 模型简单做大”。它改变了训练目标与输入特征：不强迫 draft 精确回归 Target feature，而是直接优化 token 分布；同时用多层特征融合，并在训练中模拟多步自回归时会遇到的自身预测输入。

60 秒口述版

投机解码先让廉价 draft 提 K 个 token，Target 用一次因果 forward 并行得到 K 个验证分布和下一位分布。Greedy 接受与 Target argmax 连续一致的前缀；采样模式以 $\min(1, p/q)$ 接受，首次拒绝处从 p-q 的正部做修正，因此分布无损。它在低到中 batch、memory-bound decode 时最有效。命中率没有固定百分比，EAGLE-2 论文平均每轮约 4–5.5 token，EAGLE-3 在部分任务更高；上线必须看自己的平均接受长度和 P99。EAGLE-3 主要是去掉 feature 回归约束、融合多层特征和 training-time test。

Q | 主模型和草稿模型如何生成多个 token？如何验证生成的多个 token？

面试结论

标准 Draft 模型通常自回归生成 K 次；Target 并不是再自回归 K 次，而是把 K 个草稿 token 一次性作为多 token query，在因果 mask 下并行得到各位置 logits。Medusa/MTP 可用多个 prediction heads 并行提出候选，EAGLE-2/3 可构建树；Target 用线性因果验证或 tree attention 一次验证候选路径。

4.7 不同 proposer 如何“生成多个 token”

Proposer	生成方式	Target 验证
独立小 Draft LM	顺序做 K 个廉价 autoregressive steps	把 K 个 token 一次 teacher-force verify
MTP	Target 原生多个未来 token prediction modules/heads, 可级联或并行	主干 Target 对候选做统一验证
Medusa	同一 hidden state 上多个未来位置 heads, 通常形成候选树	Tree attention / packed verification
EAGLE-2/3	轻量 feature/token draft, 自回归扩展并动态构树	Target 一次验证树, 选择最长合法路径
N-gram / suffix	从上下文或历史序列检索候选	同样走 Target verifier

4.8 Target 为什么能“一次验证多个位置”

Transformer 训练本来就是对整个 token 做 teacher forcing：每个位置的 logits 可并行计算，因果 mask 只限制依赖方向。验证时把 draft token 临时接到已确认前缀后面，Target 一次 forward 就能得到每个候选位置的 next-token distribution。只有最终接受/拒绝决策必须按前缀顺序进行，因为第 $i+1$ 个候选有效的前提是第 i 个已接受。

4.9 树验证与 KV 处理

- 把候选树节点 flatten 成一个 packed sequence，并提供 tree mask / parent index，让节点只看其祖先路径。
- Target 一次算完所有树节点 logits；Verifier 按概率或 greedy 规则找最长可接受路径。
- 候选节点的 KV 是 provisional：接受路径 commit，未选分支 discard；实现会尽量避免大规模复制。
- 树越宽，接受机会越高，但 verify tokens、KV lookahead、metadata 和 graph shape 也越大。

60 秒口述版

标准小 Draft 是顺序生成 K 个候选，但 Target 不会再做 K 次 decode，而是把这 K 个 token teacher-force 进一次因果 forward，平行得到每个位置 logits。Medusa/MTP 可以多头提出候选，EAGLE 常构建候选树；Target 用 tree attention 一次验证整棵树，然后只提交最长合法路径，未接受分支的临时 KV 丢弃。

Q | 什么是 Bonus Token？为什么要接受？

面试结论

当 K 个草稿 token 全部通过时，Target 的 verify forward 已经顺便算出了“接受完 y_K 后的下一位置”分布 p_{K+1} 。从这个 Target 分布采样/取 argmax 得到的 token 就是 Bonus Token；它不是 Draft 猜的，所以无需再做接受检验，可以直接提交。

4.10 Bonus Token 的概率语义

```
Target verify inputs: [prefix] + y1 ... yK
Target outputs: p1, p2, ... pK, p(K+1)
if y1...yK all accepted:
    z = sample from p(K+1)  # exact target token
    commit y1...yK + z
```

- 它把一次 verify 的“最后一个 logits”也利用起来，避免下一轮立即再跑一次 Target 才得到同一个位置。

- 只有所有 K 个 draft token 均接受时，这个 $p_{\{K+1\}}$ 才对应真实已确认前缀；若中途拒绝，后续位置条件不成立，不能使用该 Bonus。
- 因此一轮提交数上限为 $K+1$ ，而不是 K ；很多论文的平均接受长度/提交长度定义要确认是否含 Bonus。

一句话纠错

Bonus token 不是“第 $K+1$ 个草稿 token 被宽松接受”，而是 Target 自己生成的 token。

60 秒口述版

Bonus token 出现在 K 个草稿全被接受的情况。Target 验证 K 个候选时，一次 forward 已经得到 K 个验证 logits 和接受完第 K 个之后的下一位分布。这个下一位 token 是直接 Target 分布采样的，不是 Draft 猜的，所以无需 rejection test，可以直接提交；因此一轮最多前进 $K+1$ 个 token。

05

专家并行 (EP) 与 EPLB

全局路由、逻辑/物理专家映射、两次 A2A 和动态副本

专家并行：Global Expert Routing、两次 A2A 与 EPLB



图 5：EP 的路由与两次 All-to-All，以及 EPLB 用热门专家副本均衡物理 Rank。

Q | EP 了解吗？专家并行如何选取专家？选到专家后的流程是什么？专家负载均衡怎么做？

面试结论

Router 对每个 token 计算所有逻辑专家的 logits，经过 softmax/校正后选 Top-k expert IDs 和 gate weights；EP mapping 把逻辑 expert 映射到目标 rank/local slot。Token 按目标 rank 打包并 A2A dispatch，本地按专家分组做 MLP/Grouped GEMM，再 A2A combine 回源 rank，恢复顺序并按 gate weight 聚合。负载均衡分训练期 router loss、运行时容量/路由策略和部署期 EPLB 三层。

5.1 Router 如何选专家

```
router_logits[t, e] = W_gate · hidden[t]
(topk_weight[t, j], topk_expert_id[t, j]) = TopK(router_logits[t, :], k)
output[t] = Σ_j topk_weight[t, j] × Expert_{id[t, j]}(hidden[t])
```

- Expert ID 是模型语义里的“逻辑专家”，不是 GPU rank。
- 具体模型可能使用 softmax、sigmoid、grouped top-k、bias correction、noaux load balance、shared experts 或 routing scale。
- Top-k>1 时，一个 token 会复制到多个专家路径，最后加权合并。

5.2 选中专家后的执行流水

1. Router 产生 topk_ids / topk_weights。
2. 根据 logical→physical mapping 得到 destination EP rank 和 local expert index。
3. 统计每个目标 rank/专家 token 数，做 prefix sum，permute/pack hidden states 与元数据。
4. A2A Dispatch：把激活、expert id、source token index、route slot、可选 gate weight 发送到目标 rank。
5. 每个 rank 按 local expert 分桶，执行 fused / grouped expert MLP（通常 up/gate projection、activation、down projection）。
6. A2A Combine：把结果送回 token 的 source rank。
7. Inverse permute，按原 token 顺序和 top-k 路径还原，乘 gate weights 并求和；再进入 residual / shared expert / 后续层。

5.3 三层负载均衡

层次	机制	能解决什么	不能解决什么
训练期	aux load-balancing loss、router z-loss、group bias/noaux	让总体专家使用更均匀、路由稳定	线上实际 prompt 分布仍会偏斜
单步运行时	capacity、drop/reroute、token sorting、共享专家	限制极端峰值或提高 kernel 效率	可能改变质量；模型架构相关
部署期 EPLB	统计实际 load，重映射/复制热门专家	均衡每个物理 rank 的总工作量	不改变逻辑 Router 选择，不制造冷专家流量

60 秒口述版

EP 的 Router 先对每个 token 选 Top-k 逻辑专家并给 gate weight。系统再把逻辑 expert id 查表映射到目标 rank/local expert，按目标打包后做第一遍 A2A dispatch；各 rank 对本地专家做 grouped GEMM；第二遍 A2A combine 把结果送回源 rank，inverse permute 后按 gate weight 聚合。训练期用 router balance loss，运行期可做容量控制，部署期用 EPLB 复制热门专家或重排映射来均衡物理 rank。

Q | 多张卡场景下，门控机制、专家权重如何工作？

面试结论

Gate/Router 仍对全局逻辑专家空间打分；专家权重按 EP rank 分布，每张卡只持有自己的 local experts（以及可选冗余副本），非专家层则按 DP/TP 等方式部署。Router 输出的 global expert ID 通过 mapping table 解析为 rank、local slot 和副本选择。

5.4 逻辑专家与物理专家

概念	例子	含义
Logical expert	E3	模型 checkpoint 定义的语义专家，Router 输出这个 ID
Physical expert slot	rank2/local1	某张 GPU 上实际存储的一份 expert weights
Redundant replica	E3@rank1 与 E3@rank2	权重相同的两个物理副本，可共同承接 E3 token
Expert mapping	E3 → {rank1/local1, rank2/local0}	EPLB 可动态改变的部署映射

5.5 Gate 在多卡上放在哪里

Router 通常是 MoE 层的一小个线性投影，跟随该层的 non-expert hidden state 布局执行：在 data-parallel attention 场景中各 DP rank 对自己的 token 独立算 gate；在 TP/sequence-parallel 场景中可能先/后进行必要的 all-reduce、reduce-scatter 或 all-gather。关键不是死背“Router 一定复制”，而是保证每个 token 得到全局逻辑专家 ID，之后才进入 EP dispatch。

5.6 专家权重如何切

- 纯 EP：每个完整专家只驻留在一个 EP rank；不同专家分散到不同卡。
- EP + TP：某些系统还会在一个专家内部做 tensor parallel，但通信图更复杂；vLLM 的常见目标是用 EP 让专家完整本地化。
- Attention / dense/shared expert 权重不一定按 EP 切，可能按 TP 或 DP 复制。
- EPLB 需要可移动的完整 expert-weight layout；量化/packing backend 若不支持移动，动态重平衡可能不可用。

60 秒口述版

多卡时 Gate 仍输出全局逻辑 expert id 和 weight。每张卡只保存一部分完整专家权重，系统用 logical-to-physical mapping 把 E3 这样的逻辑 ID 转成目标 rank/local slot。若 EPLB 为 E3 建了多个副本，mapping 再选择其中负载更低的物理副本。Attention 和非专家层仍可按 TP/DP 部署，EP 只描述专家层的权重与 token 路由。

Q | 选到其他卡上的专家后，如何通过 All-to-All 发送数据？

面试结论

先根据 destination rank 统计变长 send counts，再把 token hidden states 和路由元数据按 rank/expert 连续打包；All-to-All-v 或等价高性能 backend 将不同片段发往各 rank。专家计算后使用反向 counts/layout 做 Combine，把输出送回 source rank，按 source token index 恢复顺序并聚合。

5.7 Dispatch 的数据准备

- 输入：hidden[T,H]、topk_ids[T,k]、topk_weights[T,k]。
- 展开为 $T \times k$ 条 route records；每条记录包含 source token、route slot、global/local expert、destination rank。
- 按 destination rank $\rightarrow$ local expert 排序/分桶，得到 send_counts 和 offsets。
- 复制/量化 hidden states 到连续 send buffer；元数据可显式发送，也可由固定 layout/handle 推导。

5.8 两次通信

阶段	发送方向	内容	到达后
Dispatch	source token owner $\rightarrow$ expert owner	hidden activation + routing metadata	按 local expert 组织输入
Expert compute	本地	Grouped GEMM / fused MLP	产生每条 route 的 expert output
Combine	expert owner $\rightarrow$ source token owner	expert outputs（及必要索引）	inverse permute + weighted sum

5.9 通信量与后端

$\text{one-way activation bytes} \approx T \times \text{top_k} \times \text{hidden_size} \times \text{dtype_bytes}$
 $\text{round-trip EP bytes} \approx \text{dispatch} + \text{combine} \approx 2 \times \text{above}$
 (metadata, padding, quantization and topology overhead omitted)

真实 All-to-All 可能用 allgather+reduce-scatter、NCCL alltoallv、DeepEP high-throughput/low-latency、FlashInfer NVLink、NIXL-EP、MoRI 等后端。Decode token 少、消息碎时看低延迟；Prefill token 多时看带宽和分桶效率。

5.10 如何 overlap

- 把 token 切成 microbatches/chunks，chunk 0 到达后立即计算专家，同时 chunk 1 继续通信。

- 独立 communication stream、双缓冲、persistent communication kernel。
- Router/permute/quantize 与 dispatch 融合，expert compute 与 shared expert overlap，combine 与下一段计算 overlap。
- vLLM 的 DBO 会用两个 microbatch worker 线程在 MoE yield points 处 ping-pong，让一边计算时另一边等待通信。

瓶颈判断

若每层 A2A latency 位于 critical path，先看消息规模、跨节点比例、rank load skew、SM 被通信 kernel 占用情况，再决定换后端、做 FP8 dispatch、microbatch overlap 或 EPLB。

60 秒口述版

先把 $T \times \text{top-k}$ 条路由记录按目标 rank 和 local expert 排序，计算每个 peer 的 send counts/offsets，再把 hidden activation 和索引打包进连续 buffer。第一遍 All-to-All-v 把 token 发到专家卡，本地 grouped GEMM 后，用对称的 counts/layout 做第二遍 combine，结果回源卡，按 source token index inverse permute 并乘 gate weight。单向通信量粗略是 $T \times k \times H \times \text{bytes}$ ，来回约两倍。

Q | EPLB 如何分配压力、平衡冷门专家？

面试结论

EPLB 周期性收集每个逻辑/物理专家的 token load，求一个更均衡的 logical→physical mapping；对于热门专家可创建冗余权重副本，把同一逻辑专家的 token 分流到多个 rank，并把副本放到较空闲的卡。它不是把原本属于热门专家的 token 改送给语义不同的冷门专家。

5.11 EPLB 的闭环

1. 采集：每次 forward 记录各逻辑专家/物理副本接收 token 数和 rank 总负载。
2. 平滑：在滑动 window 内聚合，避免单个 batch 噪声触发抖动。
3. 规划：选择需要复制的热门专家，计算每个物理 slot 的放置，使 max-rank load 尽量降低。
4. 迁移：把 expert weights 复制/移动到新 slot；异步模式尽量不 stop-the-world。
5. 切换：所有 rank 一致更新 mapping/version，后续 Router ID 解析到新副本。
6. 观测：检查 balancedness、A2A tail、expert GEMM imbalance、迁移 TTFT/TPOT，再决定下一轮。

5.12 “平衡冷门专家”的正确理解

不能改变模型语义

Router 选择 E3，就必须执行 E3 的权重。EPLB 可以把 E3 的副本放到原本承载冷门专家、负载较低的 Rank 上，并在多个 E3 副本间分流；不能为了均衡直接改成 E7。

因此 EPLB 平衡的是“物理卡上的总负载”，不是强行让每个逻辑专家 token 数相等。冷门逻辑专家仍然冷，只是它所在 rank 的空闲算力可被热门专家副本利用。

5.13 当前 vLLM 配置口径（随版本可能变化）

参数	当前文档默认	作用
<code>window_size</code>	1000 steps	记录专家负载的滑动窗口
<code>step_interval</code>	3000 steps	多久触发一次重平衡
<code>num_redundant_experts</code>	0	每个 EP rank 之外的额外全局专家副本预算
<code>use_async</code>	true	尽量异步迁移权重，降低停顿
<code>policy</code>	default	放置/复制策略
<code>communicator</code>	auto / null	权重迁移后端，如 NIXL、Gloo、NCCL 等

5.14 EPLB 的风险与监控

- 统计滞后：流量分布变化快时，刚迁移完就可能过时。
- 权重迁移成本：大专家权重跨卡/跨节点搬运会影响带宽与 tail latency。
- 副本占显存：冗余专家挤压 KV cache 或 batch capacity。
- 路由一致性：所有 rank 必须原子地看到同一 mapping/version，避免 token 发错。
- 拓扑感知：同节点 NVLink 与跨节点 RDMA 代价不同，均衡 token 数不等于均衡时间。
- 量化兼容：packed expert weights、scale layout 和 kernel backend 必须支持移动/复制。

60 秒口述版

EPLB 每个 forward 收集专家 token load，在一个窗口内平滑，周期性求新的逻辑专家到物理 rank 映射。热门 E3 可以复制成多个相同权重的物理副本，并放到负载较低的 rank，再把 E3 的 token 在这些副本间分流。它平衡的是每张卡总工作量，不会把 E3 token 改路由到语义不同的冷门 E7。异步搬权重后要原子切换 mapping，并监控迁移开销、KV 显存和 A2A tail。

06

面试速查与易错点

每题 30-60 秒先答到位，再根据追问下钻

01 一个请求的完整链路？

API 层模板/tokenize → Request 进入 EngineCore → Scheduler 每步给每请求分 token budget, KV Manager 查 prefix/分 block → Executor/Worker/ModelRunner 执行 prefill/decode → Sampler 产 token → detokenize/stream → 结束后释放 KV。

02 Prefill 和 Decode 是两个独立队列吗？

概念上可分阶段，但 V1 是统一 scheduler: waiting/running 请求都转成“本轮算多少 token”。同一 batch 可混合 decode 与 chunked prefill。

03 为什么必须 Prefill？

Decode 需要 prompt 的每层 K/V；Prefill 先并行计算 prompt KV 和最后位置 logits。Prefix cache 只是复用已算 KV。

04 资源不足怎么办？

先 prefix reuse、chunk prefill、等待和 watermark admission；KV 真不足时 preempt 运行请求并释放 blocks，恢复时命中最长 prefix + 重算；永远放不下则拒绝。

05 PagedAttention 三句话？

KV 预分配成固定物理 blocks；Block Table 做逻辑→物理映射；Slot Mapping 给新 token 指定写槽。CPU KV Manager 管生命周期，GPU kernel 按映射读写。

06 多个公共 block 怎么匹配？

按 parent hash + 当前 block token + extra keys 做链式 hash，从 block 0 连续查到第一个 miss，只复用最长公共前缀，不做任意集合匹配。

07 Block 被抢占后怎么恢复？

CPU 请求状态还在，GPU KV 被释放。重新 admission 时找最长仍有效 prefix，重新分配并 prefill 缺失

token，回到原生成位置继续。

08 Free 后旧数据？

映射/refcount 先失效，旧字节不可达；当前主线会对 newly allocated KV blocks 在 forward 前 GPU 清零，然后再写入。

09 Prefill 与 Decode 谁优先？

Chunked prefill 下通常先调度 decode，剩余 token budget 给 prefill，保护 ITL；请求之间仍受 FCFS/priority、KV 和公平性约束。

10 Graph mode？

CUDA Graph 捕获固定 GPU launch 序列和静态地址，replay 降低 CPU launch overhead；与 torch.compile 不同但可组合。

11 为什么 warmup？

先完成 JIT、autotune、workspace、allocator 和通信初始化，避免 capture 内发生动态行为。

12 新 graph shape 怎么办？

在 capture 范围与 key 兼容时向上 pad 到最小 capture size，dummy 行用 null block/-1 slot；超范围或不兼容则 piecewise/eager。

13 投机解码流程？

Draft 提 K 个候选，Target 一次计算 K 个验证分布和下一位；接受最长合法前缀，首次拒绝处 Target 修正；全接受时再提交 bonus。

14 投机为什么快？

小 batch decode 常 memory-bound；一次 verify 处理多个 query positions，摊薄 Target 权重读取和 launch 次数。高 batch、低接受或大 Draft 会负收益。

15 接受率多少？

没有统一值，优先报平均接受长度和端到端 speedup。EAGLE-2 论文约 4–5.5 token/cycle；EAGLE-3 某些任务更高，但必须按线上 workload 压测。

16 EAGLE 与 EAGLE-3？

EAGLE 在 feature level 自回归预测；EAGLE-3 直接优化 token distribution，融合低中高层特征，并用 training-time test 减少多步自回归偏差。

17 主模型如何一次验证多个 token?

Draft token 作为 teacher-forced 多 token query，因果 mask 下各位置 logits 可并行算；决策再从前到后接受。树候选用 tree attention。

18 Bonus token?

K 个候选全接受后，Target 已经算出的 $p(K+1)$ 直接产生下一 token；它不是 Draft token，因此无需再验证。

19 EP 流程?

Router 选 Top-k 逻辑专家 $\rightarrow$ mapping 到 rank/local expert $\rightarrow$ A2A dispatch $\rightarrow$ local grouped GEMM $\rightarrow$ A2A combine $\rightarrow$ inverse permute + gate-weight sum。

20 多卡专家权重?

每张卡持有部分完整专家或冗余副本；Router 输出 global expert ID，mapping table 解析到物理 rank/slot。

21 A2A 发什么?

Dispatch 发 hidden activation 和路由元数据；Combine 发 expert output 回 source rank。单向字节约 $T \times k \times H \times \text{dtype}$ ，来回约两倍。

22 EPLB 怎么做?

统计窗口内 load，复制热门专家并重排物理 slot，让同一逻辑专家在多个副本间分流；不改变 Router 语义，不会把 token 随便送给冷专家。

六个面试陷阱

错误说法	正确修正
“V1 有两个完全独立的 Prefill/Decode 队列。”	V1 统一按请求分配 token 数；阶段可混批，Decode 只是通常优先。
“Prefix Cache 会淘汰正在被请求使用的 block。”	活跃 block refcount>0，缓存淘汰只选零引用块；显存压力用 request preemption。
“Target 验证 K 个 token 也要做 K 次 decode。”	Target 一次 teacher-forced forward 并行得到 $K(+1)$ 个位置 logits。
“Bonus token 是宽松接受的草稿 token。”	它直接来自 Target 的第 $K+1$ 位分布。
“任何 graph 新 shape 都能 pad。”	必须在 capture 范围且 key 兼容；否则回退 piecewise/eager。
“EPLB 把热门专家 token 改送给冷专家。”	只能复制/重排相同逻辑专家权重，不能改变模型语义。

参考资料

以下资料用于核对当前 vLLM 主线实现与原始算法。实际部署请固定 vLLM commit/tag，并以对应版本源码、模型实现和硬件后端为准。

- [R1] vLLM Architecture Overview — https://docs.vllm.ai/en/latest/design/arch_overview/
- [R2] vLLM V1 User Guide / Unified Scheduler — https://docs.vllm.ai/en/latest/usage/v1_guide/
- [R3] vLLM Optimization and Tuning / Chunked Prefill — <https://docs.vllm.ai/en/stable/configuration/optimization/>
- [R4] vLLM V1 Scheduler API — <https://docs.vllm.ai/en/latest/api/vllm/v1/core/sched/scheduler/>
- [R5] vLLM Automatic Prefix Caching Design — https://github.com/vllm-project/vllm/blob/main/docs/design/prefix_caching.md
- [R6] vLLM Block Pool and Prefix Cache Source — https://github.com/vllm-project/vllm/blob/main/vllm/v1/core/block_pool.py
- [R7] Efficient Memory Management for Large Language Model Serving with PagedAttention — <https://arxiv.org/abs/2309.06180>
- [R8] vLLM KVBlockZeroer / Worker Utilities — <https://docs.vllm.ai/en/latest/api/vllm/v1/worker/utils/>
- [R9] vLLM Compilation and CUDA Graph Configuration — <https://docs.vllm.ai/en/latest/api/vllm/config/compilation/>
- [R10] vLLM CUDA Graph Dispatcher Source — https://github.com/vllm-project/vllm/blob/main/vllm/v1/cudagraph_dispatcher.py
- [R11] Fast Inference from Transformers via Speculative Decoding — <https://arxiv.org/abs/2211.17192>
- [R12] vLLM Speculative Decoding — https://docs.vllm.ai/en/latest/features/speculative_decoding/
- [R13] EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty — <https://arxiv.org/abs/2401.15077>
- [R14] EAGLE-2: Faster Inference with Dynamic Draft Trees — <https://arxiv.org/abs/2406.16858>
- [R15] EAGLE-3: Scaling up Inference Acceleration via Training-Time Test — <https://arxiv.org/abs/2503.01840>
- [R16] vLLM Expert Parallel Deployment and EPLB — https://docs.vllm.ai/en/latest/serving/expert_parallel_deployment/
- [R17] vLLM EPLB State / Logical and Redundant Experts — https://docs.vllm.ai/en/stable/api/vllm/distributed/eplb/eplb_state/
- [R18] vLLM Distributed All-to-All Interface — <https://docs.vllm.ai/en/latest/api/vllm/distributed/>
- [R19] DeepEP: High-performance MoE Dispatch and Combine — <https://github.com/deepseek-ai/DeepEP>

[R20] vLLM Dual Batch Overlap — <https://docs.vllm.ai/en/latest/design/dbo/>

最后的答题建议

把框架题回答成系统题

面试官追问到最后，通常都落在四个量：KV 显存、HBM/网络字节、GPU kernel 时间和 TTFT/ITL/P99。每个设计都说明“谁拥有状态、数据怎么走、何时回退、什么时候负收益”，答案就会从背术语变成真正做过系统。